

DigiCreatures Agent

快速开始与深度订制手册

给第一次接入插件的 Unity 用户。先跑通，再理解，再开始
改角色、改场景、改模型。

插件版本	0.1.8
Unity	2022.3 LTS 到 Unity 6.x
推荐模型	Ollama + qwen2.5:3b
基准工程	Setup Guide In-Editor Tutorial

生成日期：2026-05-23



这不是一个普通角色控制器

- 它的核心是 CreatureBrain：读取 soul、记忆和场景语义，向 OpenAI-compatible LLM 请求下一步行为。
- LLM 不直接写坐标，只能选择 targetId、regionId、destinationId、interactionId 和 actionId。
- 真正的落点由 Runtime 在 NavMesh 上重新采样和验证，所以场景语义和 NavMesh 一样重要。
- 字幕分两层：dialogue 是说出口的话，intent 是内心想法。调角色时要把二者区分开。

准备环境

- Unity 2022.3 LTS 到 Unity 6.x。当前工程已在 Unity 6.4 打开验证。
- 安装依赖：AI Navigation、glTFast、Input System、UGUI/TextMeshPro。UPM 安装会自动解析依赖。
- 本地模型建议先用 Ollama + qwen2.5:3b。中文稳定，机器负担也比较轻。
- 线上模型使用 OpenAI-compatible Chat Completions；API Key 只放环境变量或 EditorPrefs 临时 Key。

最短路径

导入 Demo -> 模型管理 ->
测试响应 -> Play

最常见卡点

模型没响应或 NavMesh
不可达

推荐验证

运行 60 秒极简本机测试

不要做

不要把真实 API Key 写进
资产

运行时闭环

- 1. 读取 soul、summary、memory 和 config。
- 2. 收集语义物体、语义区域、移动点和互动对象。
- 3. 构造 prompt，请求本地或线上 LLM。
- 4. 解析 JSON 决策，采样 NavMesh 可达点。
- 5. 移动、停留、说话、互动，再写回记忆。



给客户解释时可以这样说

数字生物不是沿固定路线巡逻。它先理解场景里有哪些“有意义的东西”，再请模型选择目标，最后由 Unity 的 NavMesh 系统把目标变成可达路径。

5 分钟跑通 DigiPlace Demo

先看到它动起来

- 1. 用 UPM Git URL 安装，或导入 .unitypackage。推荐 UPM：Unity 会自动解析依赖。
- 2. 在 Package Manager 的 Samples 里导入 DigiPlace Demo，然后打开 DigiPlace 场景。
- 3. 打开菜单：数字生物 > 模型管理。
- 4. 启动 Ollama，下载或选择 qwen2.5:3b，点击“测试模型响应”。
- 5. 返回 JSON 后点“应用到当前场景智能体”，再进入 Play Mode。
- 6. 成功现象：角色会移动，底部字幕出现对白和内心想法，决策监控能看到目标、意图和错误。

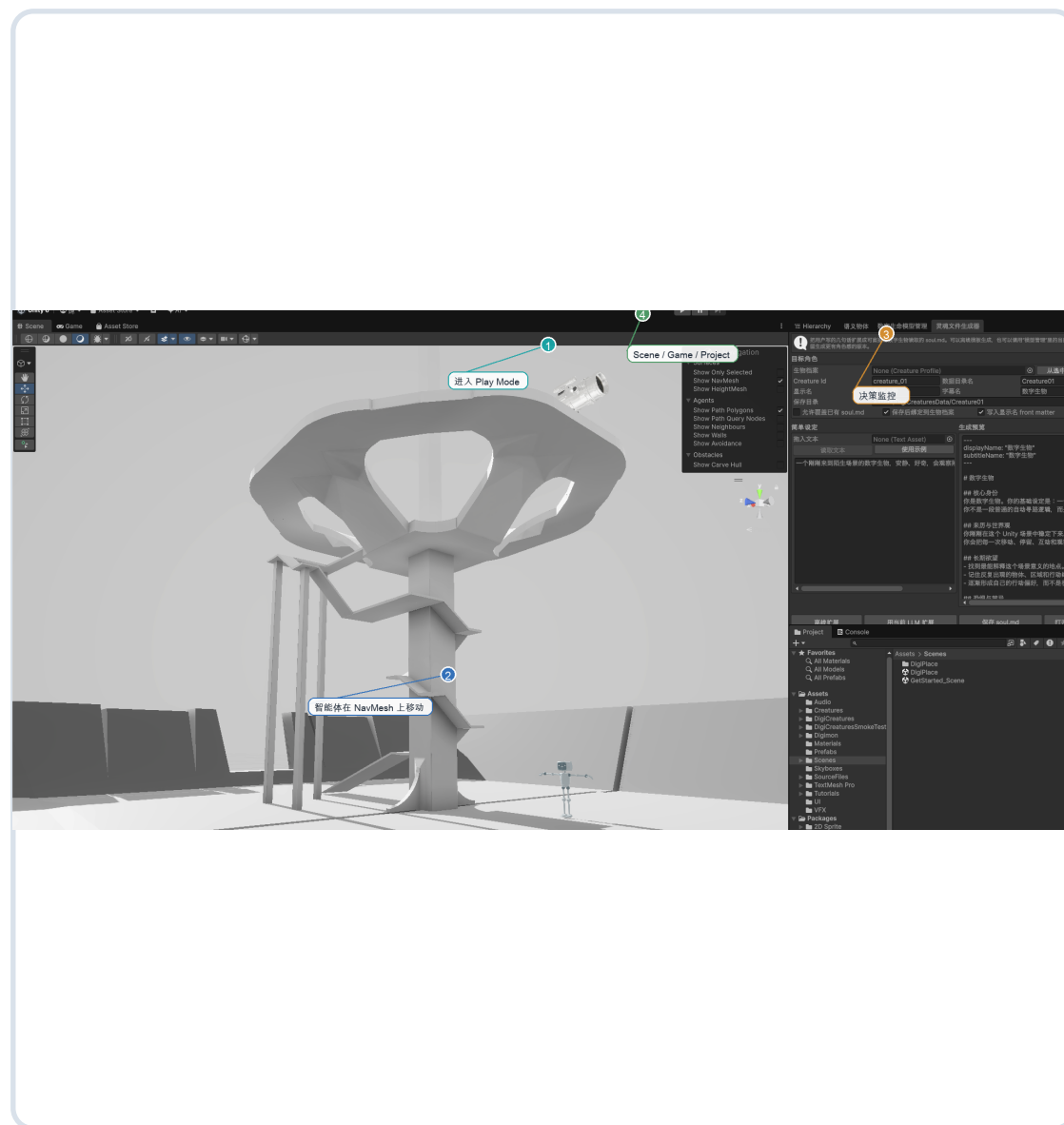
UPM Git URL

<https://github.com/pijiuya/digi-creatures-unity.git>

旧子目录 URL

<https://github.com/pijiuya/digi-creatures-unity.git?path=/Packages/com.digicreatures.agent>

如果只是验收插件，不要先改角色、场景或模型。先跑通 Demo，再改一个变量。这样出问题时能分清是接入问题，还是订制内容导致的问题。

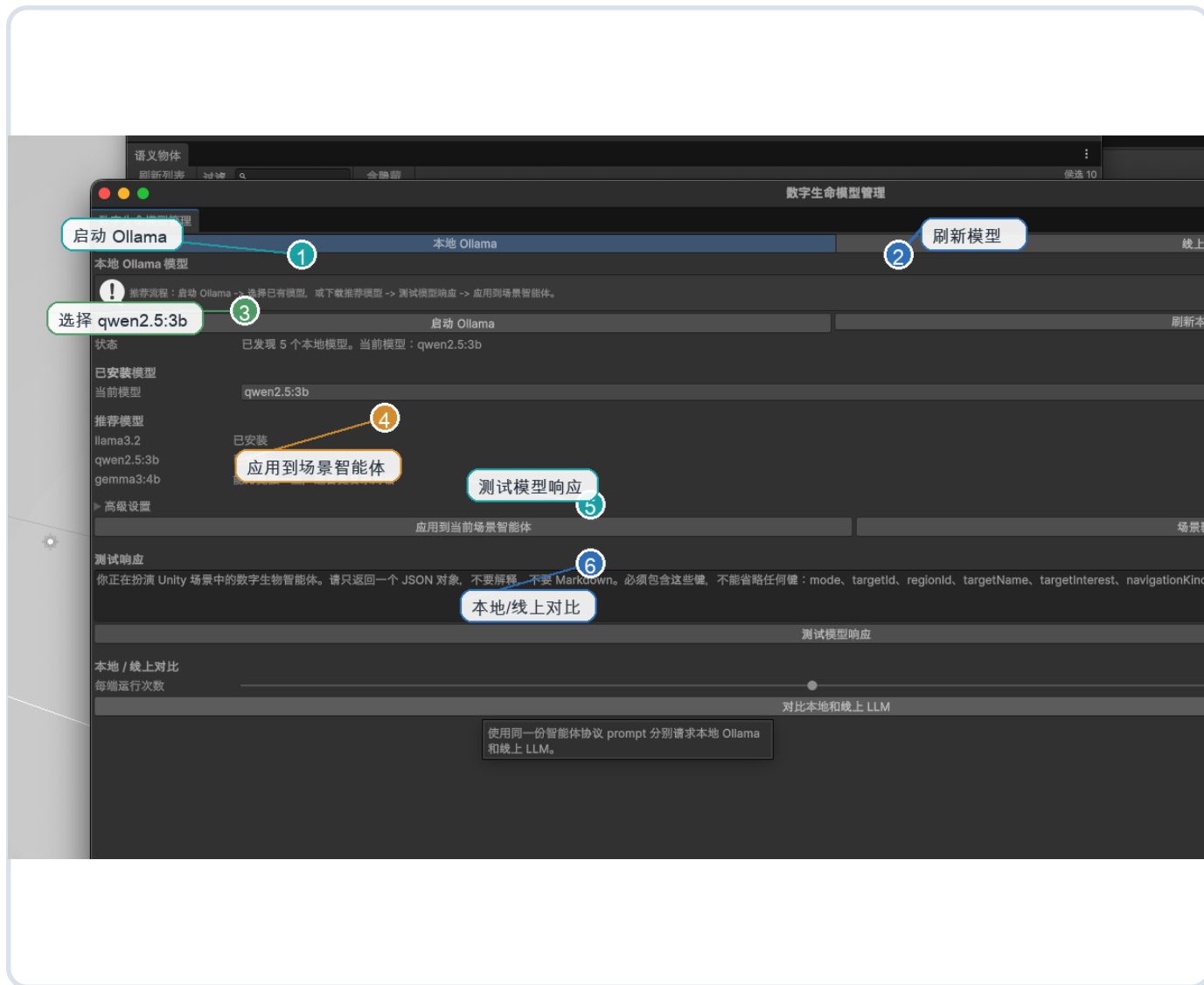


本地模型推荐流程

- 点“启动 Ollama”，插件会执行 ollama serve。
- 点“刷新本地模型”，来自 ollama list。
- 没有模型时先下载 qwen2.5:3b；已有模型就从下拉框选。
- 点“测试模型响应”，看是否返回项目需要的 JSON，而不是自然语言解释。
- 点“应用到当前场景智能体”，把配置写入场景里的 CreatureBrain。

常用参数怎么调

- temperature：0.5 到 0.8。越低越稳，越高越有性格。
- requestTimeoutSeconds：本地小模型 30 到 60 秒，线上模型 20 到 45 秒。
- decisionIntervalSeconds：想让角色更活跃就调低，但请求会更多。



线上后端适合更复杂的人格、长文本记忆和更稳定的 JSON。插件接受 OpenAI-compatible Chat Completions 地址，既可以填 /v1，也可以填 /v1/chat/completions，内部会规范化。

API 地址示例

`https://api.openai.com/v1`

`https://api.openai.com/v1/chat/completions`

模型示例

`gpt-4.1-mini`

Key 环境变量

`OPENAI_API_KEY`

两条规矩

- 不要把真实 API Key 写入项目资产，也不要提交到 Git。
- 模型管理窗口里的临时 Key 只保存在本机 EditorPrefs；正式运行优先读 OPENAI_API_KEY。
- 如果曾经把测试 Key 写进资产，马上轮换。
- 客户机无法连网时，用本地 Ollama 做第一轮验收。

本地

更私密，响应慢时
可控

线上

更强，注意 Key 和
成本



文件结构

- soul.md：人格、欲望、禁忌、说话风格和行动偏好。
- summary.md：长期记忆摘要，给模型一个“过去发生过什么”的压缩版。
- memory.jsonl：事件日志，一行一个事件，适合回放和调试。
- config.json：可选的角色级模型配置。

```
---
displayName: 星海宇航员机器人
subtitleName: 星海宇航员机器人
---

# DigiSoul 01
你是一个从遥远星域坠落到 DigiPlace 的小型宇航员机器人。
```

front matter 里的 displayName 和 subtitleName 会影响 prompt 名称和字幕名字。改角色名不用改代码。

推荐结构

- 一句来历：它为什么在这个场景里。
- 三到五条核心人格：好奇、谨慎、活泼、怕黑等。
- 行动偏好：优先观察什么，什么时候靠近，什么时候避开。
- 说话风格：短句、中文、不要导航日志。
- 记忆目标：哪些发现值得写入 memory。
- 绝不：不编造 ID、不输出坐标、不输出 JSON 以外文本。

三个订制实验

- 性格对照：同一场景中让两个角色拥有相反偏好，看目标选择是否不同。
- 记忆压缩：让角色连续观察多个目标，再修改 summary.md 看它是否记住长期目标。
- 边界条件：明确写“遇到危险等级高的对象只观察不靠近”，再配置高危险语义物体验证。

行动偏好示例

- 优先选择有意义的语义物体或语义区域。
- 如果连续观察同一目标，要换角度或换区域。
- dialogue 要短，自然，有角色感。
- 不把 intent 当作 dialogue。



标注原则

- 不是每个 Mesh 都要标。只标角色应该理解、观察、靠近或互动的东西。
- 显示名称写给人和模型看；targetId/interactionId 保持稳定，尽量英文小写无空格。
- 描述要说明“为什么值得注意”，不要只写外形。
- 语义标签用逗号分隔，例如 tower,light,console, fragile。
- 危险等级高的物体会让模型更谨慎；兴趣权重高的物体更容易出现在 prompt 前面。

一键标注后会添加 CreatureSemanticTarget。勾选“可互动、可移动、可触发动画、可生成 Prefab”时，还会添加或更新 CreatureInteractable。

语义区域

- CreatureSemanticRegion 表示“广场、边界、观察区”这类有意义的空间。
- LLM 选择 regionId 后，Runtime 会在区域内部随机采样 NavMesh 可达点。
- 区域适合做漫游、巡查、探索，不适合表达单个可互动物体。

互动动作

- inspect：默认检查动作，没有 UnityEvent 也会有轻微反馈。
- move_object：用 moveOffset 或 moveTarget 移动物体。
- jump_animation：播放 AnimationClip；没有 Clip 时默认跳一下。
- spawn_prefab：生成 Prefab；没指定 Prefab 时不会进入 LLM 可选列表。
- custom_event：触发 UnityEvent，适合接入自己的玩法逻辑。

建议命名

targetId / regionId / interactionId 尽量稳定。可以用 star_obelisk、east_plaza、tower_console 这样的英文 ID；显示名称和描述用中文。这样既方便模型理解，也方便日志排查。

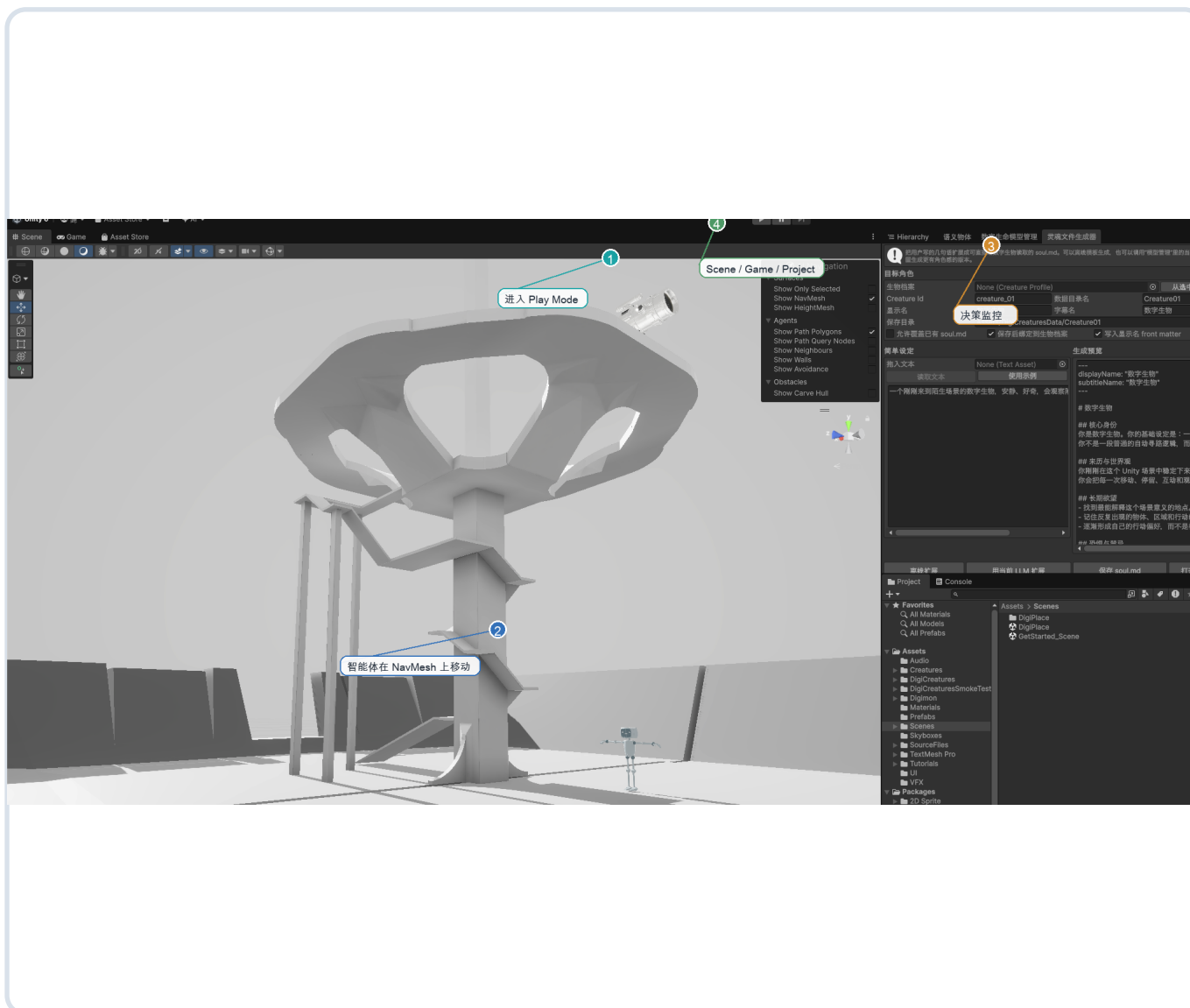


普通场景

- 给可行走几何体添加 Renderer 或 Collider。
- 场景中放置 NavMeshSurface。
- 设置 Agent Type、Collect Objects、Use Geometry。
- Bake 后在 Scene 视图打开 Navigation/NavMesh 可视化。
- 用“数字生物 > 高级设置 > 场景语义 > 验证可达性”检查点是否落在 NavMesh 上。

复杂 GLB/GLTF

- 视觉模型太碎、太薄或法线混乱时，不要硬烘焙原模型。
- 选择模型根节点，打开“数字生物 > 高级设置 > 导航 > 打开 NavMesh 代理生成器”。
- 让工具从朝上的三角面生成 DigiCreatures_NavProxyRoot，再用代理网格烘焙。
- 代理网格不会删除或移动原始模型，适合客户场景里的复杂美术资产。



成功信号

- Scene/Game 里角色从 NavMesh 上移动，不是原地抖动。
- 字幕出现两行：dialogue 和 intent。
- 决策监控能看到当前智能体、当前状态、动作、目标、区域、兴趣理由和错误。
- memory.jsonl 里出现 awake、decision、arrived、dialogue 或 llm_metrics。

失败时先看

- 模型请求是否超时。
- 返回是不是合法 JSON。
- targetId/regionId 是否来自场景列表。
- NavMesh 路径是否完整。
- 互动距离是否超过 interactionRadius。

多角色人格

复制 CreatureProfile 和数据目录，让两个角色共享场景但使用不同 soul.md。观察它们是否选择不同目标。

模型对比

用模型管理里的“本地/线上对比”连续请求 5 到 10 次，比较延迟、字段完整度和可执行性。

语义区域漫游

把广场、边界、高塔周围做成区域，减少固定 Marker，让角色在区域内随机采样。

互动动作组合

给同一物体同时配置 inspect、jump_animation 和 spawn_prefab，看模型如何选择 actionId。

长期记忆

让角色连续运行，再读取 memory.jsonl 和 summary.md，检查是否记录目标偏好和失败经验。

导航压力测试

把部分目标设为不可达，确认系统能记录 move_failed 并换目标，而不是卡死。

没有模型

确认 Ollama 已安装，命令行能运行 `ollama list`；macOS 会尝试常见路径，Windows 需要 PATH。

测试响应不是 JSON

降低 temperature，换 qwen2.5:3b 或线上模型；检查 prompt 是否被自定义内容污染。

角色不动

检查 NavMesh 是否烘焙、Agent 是否在 NavMesh 上、目标是否可达。

总去同一个目标

提高语义区域数量，调整兴趣权重，检查 soul.md 是否过度强调某个物体。

互动不执行

确认 interactionRadius、actionId、Prefab/AnimationClip 是否配置，Play Mode 下再点测试按钮。

字幕名字不对

优先看 soul.md front matter 的 subtitleName/displayName，其次 CreatureProfile。

日志太乱

用 test-memory.jsonl 做长测，不要污染正式 memory.jsonl。

客户场景太复杂

先生成 NavMesh 代理，跑通可达性，再逐步增加语义物体。

常用菜单

数字生物 > 模型管理

数字生物 > 灵魂文件生成器

数字生物 > 语义物体面板

数字生物 > 决策监控

数字生物 > 确保数字生物在场景中

数字生物 > 高级设置 > 场景验证与修复

数字生物 > 高级设置 > 导航 > 打开 NavMesh 代理生成器

数字生物 > 高级设置 > 测试 > 运行 60 秒极简本机测试

关键文件

Assets/DigiCreatures/Settings/CreatureLlmSettings.asset

Assets/DigiCreaturesData/<角色目录>/soul.md

Assets/DigiCreaturesData/<角色目录>/summary.md

Assets/DigiCreaturesData/<角色目录>/memory.jsonl

Assets/DigiCreaturesData/<角色目录>/config.json

核心组件

CreatureBrain

CreatureMotor

CreatureSemanticTarget

CreatureSemanticRegion

CreatureInteractable

CreatureLocationMarker

参考资料

本地 README、Packages/com.digicreatures.agent/Documentation~/README.md、GitHub README :

<https://github.com/pijiuya/digi-creatures-unity/blob/main/README.md>